

This is a conversation I have had with chat gpt where we brainstormed everything get the idea of what we are building from this

So i want to build a pos system for a restaurant called glitos. Now i know exactly how they operate where when a customer comes in they write their irder in a receipt then the one in the couter passes it to the server who passes the food to the waiter. At the end of the day they tally the days sales in a book to know the total amount of money sold. So i want to build it and i want it to be pwa system. Ask me questions about the business to help me plan and build it

Since you already understand how they work, don't start by designing screens. Start by mapping the business. A POS that matches their workflow will be much easier for them to adopt than forcing them to change how they work.

I'll ask the questions in phases.

## 1. Restaurant Basics

- Is it dine-in only, takeaway only, or both?
  - Do they do deliveries?
  - How many branches are there?
  - About how many customers do they serve per day?
  - How many employees use the system?
- 

## 2. Staff Roles

Who exactly works there?

Example:

- Cashier
- Waiter
- Kitchen staff
- Manager
- Owner

For each role:

- What are they allowed to see?
  - What are they allowed to edit?
- 

## 3. Current Ordering Process

Walk me through one customer.

Example:

Customer enters →

Cashier writes receipt →

Customer pays? or pays later? →

Receipt goes to kitchen →  
Kitchen prepares food →  
Waiter serves →  
Order completed.

Questions:

- Does the customer pay before food is prepared or after eating?
  - Does the cashier keep one copy and kitchen another?
  - Are receipt books numbered?
  - Can an order be changed after it reaches the kitchen?
  - Can an order be cancelled?
- 

## 4. Menu

- How many food items are there?
  - Do prices change often?
  - Can one order contain multiple items?
  - Are there sizes? (Small, Medium, Large)
  - Are there add-ons? (Extra cheese, extra chips)
- 

## 5. Payments

What payment methods do they accept?

- Cash
- M-Pesa
- Card
- Mixed payment

Questions:

- Do they print receipts?
  - Is an M-Pesa message enough?
  - Can customers pay partly in cash and partly by M-Pesa?
- 

## 6. Kitchen Workflow

When the kitchen receives an order:

- Is food prepared in order?
  - Is priority ever changed?
  - Can multiple cooks prepare one order?
  - How do they know an order is ready?
-

## 7. Waiters

- Do waiters have assigned tables?
  - Can one waiter serve multiple tables?
  - Can multiple waiters work on one order?
- 

## 8. Tables

- How many tables?
  - Are table numbers used?
  - Can customers move tables?
  - Can two tables be combined?
- 

## 9. End of Day

Currently they total sales in a book.

Questions:

What else do they calculate?

- Cash received
  - M-Pesa received
  - Expenses
  - Profit
  - Number of customers
  - Best-selling item
- 

## 10. Inventory

This is a big one.

Do they currently track:

- Chicken pieces
- Chips
- Soda
- Cooking oil
- Flour
- Packaging

Or do they only count sales?

---

## 11. Owner Reports

If the owner opens the app tonight, what should they immediately see?

Example:

- Today's sales
  - Orders today
  - Cash vs M-Pesa
  - Best-selling food
  - Busy hours
  - Staff performance
  - Low stock
- 

## 12. Devices

Since you're making a PWA:

- Will everyone use one computer?
  - Will the cashier have a laptop?
  - Will the kitchen have a tablet?
  - Will waiters have phones?
  - Will everything run over local Wi-Fi?
- 

## 13. Internet

- Does the restaurant always have internet?
  - Should the POS continue working offline?
  - When internet returns, should it sync automatically?
- 

## 14. Future Features

Which of these do you eventually want?

- Customer loyalty points
- QR code menu
- Online ordering
- Reservations
- Kitchen display screen
- SMS notifications
- M-Pesa STK Push integration
- Expense tracking
- Payroll
- Inventory management
- Multiple branches

- Owner dashboard from home
- 

## 15. Technical Questions

Since you'll build it yourself:

- Frontend: HTML/CSS/JS or React?
  - Backend: Node.js/Express?
  - Database: SQLite, PostgreSQL, or something else?
  - Authentication: username/password or PIN?
  - Will it be hosted online or run on a local server inside the restaurant?
- 

### One suggestion before you write any code

Think of the POS as **five connected apps** rather than one:

1. **Cashier App** – creates orders, takes payments, prints receipts.
2. **Kitchen Display** – shows incoming orders, marks them as preparing and ready.
3. **Waiter App** – sees ready orders and delivers them.
4. **Manager Dashboard** – sales, reports, inventory, staff.
5. **Owner Dashboard** – view everything remotely from any device.

If you build it this way, you can start with just the cashier app and gradually add the others without rewriting the system.

My first question is this: **Describe exactly what happens from the moment a customer walks into Glitos until they leave. Include every step, even small ones like who writes the receipt, who keeps it, when payment happens, and who hands the food to the customer.**

They do both. Yes. One. A lot. 4. so i want the organization they already have to stay but they should use receipts from a small thermo printer. Customers pay after they have eaten also some are given credit. Like 20 food items. An order can contain multiple items. No size difference like large etc no addons. Cash and mpesa. I want an stk push and they can pay partly in cash and mpesa. No assigned sits for waiters food is already made so just serving and packing. Since they do delivery i want there to be a booking or ordering app for the customers who want delivery. At the end of the day they calculate everything. Do an inventory trucking. The owner should be able to see all those things. They will use their phones run over wifi. Always have internet but when it is down it will save locally and update when online. Will use mern i will have all the 5 connected apps

That's a smart decision. **Don't build KRA eTIMS into version 1.** Get the restaurant using the system first, then add eTIMS once the core POS is stable.

I would structure the project in phases.

## Phase 1 – Core POS (MVP)

- User login and roles
- Menu management
- Create and edit orders
- Thermal receipt printing
- Kitchen/serving workflow
- Cash, M-Pesa, mixed payments, and credit
- Inventory deduction
- End-of-day reports
- Offline support with automatic sync
- Owner dashboard

This alone will replace their paper workflow.

## Phase 2 – Customer Experience

- Customer delivery app/PWA
- Order tracking
- STK Push payments
- WhatsApp order confirmations
- Delivery management

## Phase 3 – Business Intelligence

- Advanced inventory
- Supplier management
- Expense tracking
- Profit analysis
- Customer loyalty
- Analytics and trends

## Phase 4 – KRA eTIMS

At this stage, every completed sale can be submitted to eTIMS.

Design your backend now so adding eTIMS later doesn't require rewriting everything. A good approach is to have a separate **Tax Service** or **Fiscal Service** in your backend.

For example:

```
Order Completed
  ↓
Payment Recorded
  ↓
Receipt Generated
  ↓
Tax Service
  ↓
eTIMS (enabled later)
```

Initially, the Tax Service can simply do nothing after generating the receipt. Later, you connect it to the KRA API. That way, the rest of your application doesn't need to change.

One more suggestion: create an **audit log** from the beginning. Every important action should be recorded, such as:

- Who created an order
- Who edited it
- Who cancelled it
- Who accepted a payment
- Who changed inventory
- When a receipt was reprinted

This becomes invaluable if there's ever a dispute about sales, stock, or cash shortages.

From what you've described, this isn't just a POS anymore—it's shaping up to be a full restaurant management system that can later support multiple restaurants with configurable workflows. If you design the data model carefully now, Glitos can be your first deployment and your test case before expanding to other businesses.

This is what we are building

i want this project to have a ui like glovo where there is alot of animations brand colors random shapes and the front page of the order website have this food category orbit g23 curves .LottieFiles animations etc design everything also make it customer friendle good ui and ux with few processes before .search forthe best design ractices for professionals successfull apps like apple google nothing tesla etc

Building a customer-facing delivery PWA with a Glovo-style aesthetic, fluid G2/G3 curves, and world-class UX is an incredible vision for Phase 2 of the Glitos system. Since the goal is a seamless, highly animated experience that competes with top-tier apps, we need to blend Glovo's vibrant playfulness with the strict, functional design rules of companies like Apple and Tesla.

Building a full-stack MERN application of this scale while balancing school requires a highly efficient toolkit. Let's break down exactly how to architect the UI/UX, the animations, and the exact design practices to apply.

## 1. The Visual Identity: "Glovo, but Premium"

The reference image you provided relies heavily on organic shapes, warm colors, and a playful layout.

- **The Food Orbit (Hero Section):** The "order orbit" image features a central focal point with secondary categories circling it. In React, you can build this using framer-motion to animate the satellites floating in, or leverage pre-built Tailwind-compatible components from libraries like *Aceternity UI* or *Sera UI*, which have ready-made responsive orbiting layouts.

- **G2/G3 Continuous Curves:** You mentioned "g23 curves." Standard CSS border-radius creates abrupt, mechanical corners (G1). Apple uses G2 (curvature continuous) and G3 (acceleration continuous) curves—often called "squircles"—which look vastly more organic and premium. In CSS, you can achieve this high-end look using `mask-image: paint(squircle)` (via CSS Houdini) or by wrapping your containers in custom SVG masks instead of relying on standard CSS rounding.
- **3D Isometric Assets:** The orbit relies on beautiful, sticker-like 3D food illustrations. To create these rapidly, generating isometric food concepts in tools like Recraft AI and then stripping the backgrounds using Clipdrop will give you perfect, high-quality transparent PNGs that float effortlessly inside your UI.

## 2. UX Principles from the Tech Giants

To make Glitos feel like a multi-million dollar app, borrow specific philosophies from these industry leaders:

- **Tesla (Zero-Friction / Single Screen):** Tesla's UI is famously flat; everything essential is one tap away. Apply this to your ordering flow. Do not force customers through a tedious multi-page checkout. They tap a food item in the orbit, a G2-curved bottom sheet slides up, they confirm the quantity, and hit a single "Pay with M-Pesa" button to trigger the STK Push.
- **Apple (Fluid Motion & Spring Physics):** Modern micro-interactions use spring physics, not linear animations. When a user taps a category in your orbit, it shouldn't just change color; it should physically depress and spring back. Use framer-motion's spring configurations (mass, stiffness, damping) to make the app feel alive and tactile.
- **Nothing (Tactile Minimalism):** Nothing OS is brilliant at contrast. While Glovo is saturated with yellow, keep your actual app background stark white (or pitch black in dark mode). Let the vibrant Glitos brand colors and the vivid food illustrations carry the visual weight. Use bold, character-heavy typography for your headers to establish brand identity.
- **Google Material 3 (Expressiveness):** Use large, thumb-friendly touch targets. Your buttons and cards should be highly accessible and leave plenty of breathing room (padding) so the interface never feels cluttered.

### 3. Integrating LottieFiles & Animations

Because your backend will be handling order statuses—from the moment the cashier creates the order to when the kitchen marks it ready—Lottie animations are the perfect way to communicate these states visually without dense text.

- **Cart Micro-interactions:** When a user adds an item, trigger a tiny Lottie confetti pop or a bounce effect on the floating cart icon.
- **Empty States & Loading:** If the user has no active orders, show a playful Lottie animation of a sweeping broom or an empty plate.
- **Order Tracking:** When the KRA/Tax Service and receipt generation are processing in the background, use a smooth Lottie loop of a pan sizzling or a delivery bike driving to keep the user entertained while they wait for the "Order Confirmed" screen.

### 4. The "Few Processes" User Flow

You emphasized having a customer-friendly flow with very few steps. Here is the ideal zero-friction architecture for the Glitos PWA:

1. **Launch:** App loads instantly (PWA cached offline support). The Glovo-style Food Orbit animates in.
2. **Select:** User taps "Chips & Chicken".
3. **Customize:** A smooth bottom sheet slides up. Since there are no complex add-ons or size differences, this is just a simple quantity selector (+/-).
4. **Checkout:** User taps "Order Now". The sheet expands slightly to ask for their phone number and offers the "M-Pesa STK Push" or "Pay Cash" options.
5. **Done:** The STK push hits their phone. Once paid, the screen transitions to a beautiful Lottie success animation and shows their order number.

Structuring a multi-app ecosystem in a MERN stack requires aggressive modularity, similar to architecting a local community marketplace like Chuka Live or a custom operations dashboard like Style by Gathoni. To keep this manageable, we will map out a unified React frontend monorepo where the routing dictates which "app" the user sees based on their login role, alongside the standalone Customer PWA.

Here is the exact, detailed design architecture and component structure for all five connected apps, utilizing Framer Motion for the fluid physics, LottieFiles for micro-interactions, and G2 continuous curves for the premium Glovo/Apple aesthetic.

# Global React Architecture Structure

## Plaintext

```
src/
├── assets/
│   ├── lottie/      # JSON files for Lottie animations (success, loading, empty cart)
│   └── icons/       # 3D isometric food icons (Recraft AI -> Clipdrop)
├── components/
│   ├── ui/          # Reusable G2 curved buttons, bottom sheets, squircle cards
│   └── animations/   # Framer Motion wrapper components
├── apps/            # The 5 distinct user interfaces
│   ├── CustomerPWA/
│   ├── CashierPOS/
│   ├── KitchenKDS/
│   ├── WaiterApp/
│   └── AdminDashboard/
├── context/         # Global state (CartContext, AuthContext, OfflineSyncContext)
└── utils/           # Thermal printer logic, eTIMS staging, STK Push handlers
```

## 1. Customer Delivery PWA (The "Glovo/Tesla" Experience)

**Design Philosophy:** Highly animated, organic shapes (G2 curves), zero friction. The background is stark white or dark grey (Nothing OS style), letting the vibrant 3D food illustrations pop.

- **OrbitHero.jsx:** The front page features the circular food category orbit. Built using framer-motion, the central node is the Glitos logo, and the categories (e.g., "Chicken", "Chips", "Combos") orbit around it.
- **ProductBottomSheet.jsx:** When a food item is tapped, instead of navigating to a new page, a bottom sheet slides up with spring physics. Since there are no sizes or add-ons, this simply contains a large image, a quantity incrementer (+/-), and a massive, G2-curved "Add to Cart" button.
- **CheckoutFlow.jsx:** A single-screen checkout (Tesla philosophy). It requires only the delivery location and a phone number.
- **PaymentModal.jsx:** Offers M-Pesa STK Push. While waiting for the user to enter their PIN on their phone, a Lottie animation of a spinning phone or loading bar plays.
- **OrderTracking.jsx:** A beautiful status page using Lottie animations that transition as the Kitchen App updates the backend (e.g., Pan sizzling -> Waiter packing -> Delivery bike driving).

## JavaScript

```
// Example Component Structure for Customer PWA
<CustomerLayout>
  <TopNav /> // Profile, Location, Cart Icon with Lottie bounce effect
  <FramerMotionOrbit> // The Glovo-style hero section
    <OrbitItem icon={Chicken3D} label="Chicken" onClick={openBottomSheet} />
    <OrbitItem icon={Chips3D} label="Chips" onClick={openBottomSheet} />
  </FramerMotionOrbit>
  <AnimatePresence>
    {isSheetOpen && (
      <ProductBottomSheet
        item={selectedItem}
        curve="G2" // custom prop mapping to mask-image: paint(squircle)
      />
    )}
  </AnimatePresence>
</CustomerLayout>
```

## 2. Cashier POS (The Fast-Paced Core)

**Design Philosophy:** High contrast, massive touch targets, extreme speed. The cashier does not need playful animations; they need efficiency to handle the line.

- **MenuGrid.jsx:** The left 70% of the screen. Since there are only about 20 items, they are displayed as a clean, responsive CSS Grid of squircle cards. Tapping instantly adds to the cart without pop-ups.
- **CartSidebar.jsx:** The right 30% of the screen. Lists the current items, the running total, and buttons for "Credit", "Cash", or "M-Pesa".
- **MixedPaymentModal.jsx:** A specific pop-up allowing the cashier to input partial cash and trigger an STK push for the remainder.
- **ThermalReceiptTrigger.jsx:** A hidden utility component that automatically formats the text payload and sends it to the local thermal printer via the Web Serial API or a local node server once payment is confirmed.

### JavaScript

```
// Example Component Structure for Cashier POS
<CashierLayout>
  <GridArea>
    {menuItems.map(item => <FastTapCard key={item.id} item={item} />)}
  </GridArea>
  <SidebarArea>
    <OrderList items={cartItems} />
    <TotalTally total={calculateTotal()} />
    <PaymentOptions>
      <SquircleButton mode="Cash" />
    </PaymentOptions>
  </SidebarArea>
</CashierLayout>
```

```

        <SquircleButton mode="M-Pesa" />
        <SquircleButton mode="Mixed" onClick={openMixedModal} />
    </PaymentOptions>
</SidebarArea>
</CashierLayout>

```

### 3. Kitchen Display System (KDS)

**Design Philosophy:** Utilitarian, color-coded, readable from 6 feet away on a tablet.

- **KanbanBoard.jsx:** The entire app is a horizontal scroll of order tickets.
- **OrderTicket.jsx:** Displays the order number, time elapsed (turns red if taking too long), and the list of items.
- **StatusActions.jsx:** Massive, full-width buttons at the bottom of each ticket. The kitchen taps "Mark Ready," which visually removes it from the KDS, alerts the Waiter App, and updates the Customer PWA.

JavaScript

```

// Example Component Structure for Kitchen KDS
<KitchenLayout>
  <Header>
    <h2>Incoming Orders <PulseLottieIndicator /></h2>
  </Header>
  <TicketContainer>
    {activeOrders.map(order => (
      <OrderTicket key={order.id}>
        <Timer startedAt={order.timestamp} />
        <ItemList items={order.items} />
        <SwipeToReadyButton onSwipe={() => updateStatus(order.id, 'READY')} />
      </OrderTicket>
    ))}
  </TicketContainer>
</KitchenLayout>

```

### 4. Waiter/Packer App

**Design Philosophy:** Mobile-first, checklist style. Since waiters do not have assigned tables and only pack/serve already made food, this app is a glorified task list.

- **ReadyFeed.jsx:** A vertical feed of orders marked "Ready" by the kitchen.
- **PackingChecklist.jsx:** Expanding an order shows a checklist. The waiter packs the chips and chicken, checks them off, and taps "Served/Dispatched".

JavaScript

```

// Example Component Structure for Waiter App
<WaiterMobileLayout>

```

```

<LottieBellAnimation isActive={newReadyOrder} />
<ReadyOrdersList>
  {readyOrders.map(order => (
    <CollapsibleCard key={order.id}>
      <OrderSummary number={order.id} type={order.isDelivery ? "Delivery" : "Dine-in"} />
      <ActionTap buttonText="Mark as Served" onClick={() => clearOrder(order.id)} />
    </CollapsibleCard>
  ))}
</ReadyOrdersList>
</WaiterMobileLayout>

```

## 5. Owner/Manager Dashboard

**Design Philosophy:** Apple Health style analytics. Clean charts, data-dense but readable, heavy use of white space.

- **DailyTallyOverview.jsx:** The immediate view showing Total Sales, Total M-Pesa, Total Cash, and Total Credit.
- **InventoryTracker.jsx:** Visual progress bars showing stock levels (e.g., Chicken pieces, Cooking oil, Packaging). If stock drops below a threshold, it turns red.
- **AuditLogView.jsx:** A searchable data table detailing every action (who cancelled an order, who took a payment) to prevent cash shortages.

### JavaScript

```

// Example Component Structure for Admin Dashboard
<AdminLayout>
  <SidebarNav links={['Overview', 'Inventory', 'Audit Log', 'Settings']} />
  <MainContent>
    <MetricGrid>
      <DataCard title="Total Sales" value={totalSales} trend="+12%" />
      <DataCard title="M-Pesa / Cash Split" chart={<DonutChart data={splitData} />} />
    </MetricGrid>
    <InventorySection>
      {inventoryItems.map(item => (
        <StockLevelBar item={item.name} current={item.count} max={item.capacity} />
      ))}
    </InventorySection>
  </MainContent>
</AdminLayout>

```

## Technical Implementation for the Premium UI

1. **G2/G3 Squircles:** Standard CSS border-radius: 24px looks cheap. To get the premium continuous curve, create a reusable wrapper component in React using CSS clip-path or SVG masks.

## CSS

```
.squircle-g2 {  
  mask-image: paint(squircle);  
  --squircle-radius: 24px;  
  --squircle-smooth: 1; /* Pushes the curve to G2 */  
}
```

2. **Lottie Integration:** Use lottie-react for the animations. Keep the JSON files incredibly small (under 50kb) so the PWA loads instantly.
3. **Framer Motion Physics:** For the bottom sheets and the Glovo orbit, use spring configurations instead of linear ease-in-out.

## JavaScript

```
const springConfig = { type: "spring", stiffness: 300, damping: 20 };  
<motion.div transition={springConfig}> ... </motion.div>
```